

PnP và ước lượng tư thế (Perspective-n-Point)

PHỤ THUỘC A/01-mô-hình-camera-va-distortion (mô hình chiếu và ma trận K — PnP giả định cả hai đã biết chính xác);
A/02-homography-va-hình-học-phang (phân rã homography, một trong các cách khởi tạo).

TÓM TẮT

Chương này dựng cỗ máy trung tâm của cả cây: cho một tập điểm 3D đã biết toạ độ trong hệ vật và hình chiếu của chúng trên ảnh, tìm pose của camera. Bức tranh gồm hai tầng và tách bạch chúng là điều quan trọng nhất cần mang ra khỏi chương. Tầng thứ nhất là các lời giải *dạng đóng* — P3P, EPnP, DLT, IPPE — chúng nhanh, không cần khởi tạo, và *không* tối ưu theo tiêu chí có ý nghĩa. Tầng thứ hai là *tinh chỉnh phi tuyến* cực tiểu hoá sai số tái chiếu, và đây mới là nghiệm cuối cùng. Ba điều thực hành: với target phẳng hãy dùng IPPE vì nó cho cả hai nghiệm; với nhiều tag hãy chạy *một* PnP trên toàn bộ tập góc chứ đừng trung bình pose; và hãy trọng số hoá theo chất lượng từng góc thay vì coi mọi góc như nhau. Nếu chỉ nhớ một điều: *lời giải dạng đóng là để khởi tạo, tinh chỉnh mới là kết quả* — và Jacobian của bước tinh chỉnh chính là ma trận mà A/05 đảo để ra mọi công thức sai số của cây.

KIẾN THỨC CẦN CÓ TRƯỚC

- **PnP — Perspective-n-Point**: bài toán tìm pose camera từ n điểm 3D đã biết và hình chiếu 2D của chúng, với K đã biết. Ba điểm là số tối thiểu cho một tập nghiệm hữu hạn; bốn điểm thường đủ để chọn ra một nghiệm.
- **sai số tái chiếu (reprojection error)** — với một pose giả định ξ , chiếu điểm 3D ra ảnh được $\hat{u}_i(\xi)$, rồi lấy hiệu với góc đo được u_i . Đây là hàm mục tiêu có ý nghĩa vật lý vì nó đo đúng đại lượng mà nhiễu tác động lên — toạ độ pixel.
- **sai số đại số (algebraic error)** — đại lượng mà các lời giải tuyến tính như DLT cực tiểu hoá, thường có dạng $\|Ah\|$. Nó dễ giải vì tuyến tính, và nó *không* tỉ lệ với sai số tái chiếu — đó là toàn bộ lý do cần bước tinh chỉnh.
- **Gauss-Newton** — phương pháp lập cho bình phương tối thiểu phi tuyến: tuyến tính hoá phần dư quanh ước lượng hiện tại, giải một hệ tuyến tính cho bước cập nhật, lặp lại. Nó xấp xỉ Hessian bằng $J^T J$, bỏ số hạng chứa đạo hàm bậc hai của mô hình.
- **Levenberg-Marquardt (LM)** — Gauss-Newton có thêm số hạng chính quy hoá λI trên đường chéo, nội suy giữa Gauss-Newton (khi λ nhỏ) và bước giảm gradient (khi λ lớn). Nó bền hơn khi khởi tạo xa hoặc khi $J^T J$ kém điều kiện.
- **tham số hoá tối thiểu của xoay** — pose có sáu bậc tự do nhưng R có chín phần tử với sáu ràng buộc. Để tối ưu trên $SO(3)$ cần một tham số hoá cục bộ sáu tham số — thường là vectơ xoay qua ánh xạ mũ, tức $\mathfrak{se}(3)$. Dùng góc Euler thay vào là nguồn của kỳ dị gimbal lock.
- **RANSAC — Random Sample Consensus**: chọn ngẫu nhiên tập con tối thiểu, giải, đếm số điểm phù hợp, lặp và giữ tập con tốt nhất [1]. Nó loại điểm ngoại lai chứ không cải thiện độ chính xác của điểm nội lai.

CÂU HỎI MỞ ĐẦU

- Q1.** P3P với ba điểm cho tối đa bốn nghiệm hình học hợp lệ. Vì sao bốn mà không phải hai hay tám, và điều đó có ý nghĩa gì với việc bạn phải làm ở tầng code?

- Q2. EPnP chạy nhanh, không cần khởi tạo, xử lý được hàng trăm điểm. Vì sao nó vẫn không đủ, và cụ thể đại lượng nào nó cực tiểu hoá mà không phải đại lượng bạn quan tâm?
- Q3. Bạn có bốn tag, mỗi tag bốn góc. Có hai cách: bốn lần PnP rồi gộp, hoặc một lần PnP trên mười sáu góc. Chúng cho kết quả khác nhau đáng kể — vì sao, và cách nào đúng?
- Q4. Bộ tối ưu của bạn dùng góc Euler để tham số hoá xoay. Nó chạy tốt trong thử nghiệm rồi thỉnh thoảng phân kỳ hoàn toàn ở một số tư thế. Cơ chế gì, và cách sửa đúng là gì?
- Q5. Một tag ở xa và nghiêng mạnh, một tag ở gần và nhìn thẳng, cả hai đều nhìn thấy. Nếu bạn cho chúng trọng số như nhau trong PnP hợp nhất, bạn đang giả định gì — và giả định ấy sai theo chiều nào?

1.1 Đặt vấn đề

Toàn bộ cây này quy về một phép biến đổi: từ vài chục tọa độ pixel sang sáu con số pose. Chương này là chương mô tả phép biến đổi ấy.

Bài toán có một lịch sử dài hơn thị giác máy tính. Trường hợp ba điểm được Grunert giải năm 1841 trong bối cảnh trắc địa — xác định vị trí một trạm quan trắc từ ba mốc đã biết — và phân tích đầy đủ về số nghiệm của nó được Haralick và cộng sự tổng hợp lại một thế kỷ rưỡi sau [2]. Trường hợp n điểm tổng quát thì có nhiều lời giải hiện đại, mỗi lời giải đánh đổi khác nhau giữa tốc độ, độ bền và độ chính xác.

Điều đáng nói với người làm marker — và lý do chương này không chỉ là một danh mục thuật toán — là một sự nhầm lẫn rất phổ biến về *vai trò* của các thuật toán ấy. Người ta hay hỏi “dùng PnP nào tốt nhất”, như thể đó là một lựa chọn duy nhất. Câu hỏi ấy đặt sai, vì bức tranh đúng có hai tầng và chúng phục vụ hai mục đích khác nhau.

Tầng thứ nhất gồm các lời giải *dạng đóng*: P3P, EPnP, DLT, IPPE, SQPnP. Chúng giải một hệ đại số và cho ra nghiệm trong một số hữu hạn phép tính, không cần điểm khởi đầu. Ưu điểm rõ ràng; nhược điểm thì kín đáo hơn: chúng cực tiểu hoá một đại lượng đại số nào đó *không phải* sai số tái chiếu. Hệ quả là nghiệm chúng cho ra nói chung không phải nghiệm hợp lý cực đại, và độ lệch ấy đáng kể khi nhiễu không nhỏ.

Tầng thứ hai là *tinh chỉnh phi tuyến*: xuất phát từ một pose khởi tạo, lặp Gauss–Newton hoặc LM để cực tiểu hoá sai số tái chiếu. Đây là bước cho ra nghiệm hợp lý cực đại dưới giả thiết nhiễu Gauss, và đây là nghiệm duy nhất mà công thức hiệp phương sai của $A/05$ áp dụng được. Bỏ bước này là bỏ đi cả độ chính xác lẫn khả năng nói gì đó có cơ sở về bất định.

Cách hiển nhiên mà nhiều cài đặt thực tế làm — gọi một hàm PnP dạng đóng rồi dùng thẳng kết quả — do đó hỏng theo hai cách cùng lúc. Nó mất một phần độ chính xác, và nó mất luôn ma trận thông tin mà bước tinh chỉnh sinh ra như một sản phẩm phụ miễn phí. Cái thứ hai đắt hơn cái thứ nhất, vì không có ma trận thông tin thì không có trọng số để hợp nhất, không có cơ sở để loại ngoại lai theo chi-square, và không có gì để đưa vào ngân sách sai số.

Có một cách hiển nhiên thứ hai cũng hỏng, và nó phổ biến hơn: khi có nhiều tag, chạy PnP cho từng tag rồi gộp các pose lại. $A/05$ mục 7 đã nêu kết luận; mục 6 của chương này giải thích cơ chế bằng ngôn ngữ của bài toán tối ưu.

TRƯỚC GIÁC

Hình dung bài toán PnP như việc định vị bằng giác kế.

Bạn đứng ở một chỗ chưa biết và nhìn thấy vài cột mốc mà bạn biết chính xác toạ độ. Với mỗi cột mốc, bạn đo được *hướng* tới nó — không phải khoảng cách, chỉ hướng. Câu hỏi: bạn đang ở đâu và đang quay mặt về phía nào?

Mỗi cột mốc cho bạn một tia: “từ chỗ tôi đứng, cột mốc này nằm theo hướng đó”. Với một cột mốc thì vô số vị trí thoả mãn. Với hai thì bớt đi. Với ba thì — và đây là chỗ bất ngờ — vẫn còn tới bốn vị trí có thể, vì bài toán đại số ẩn sau nó là bậc bốn. Phải có cột mốc thứ tư để loại bớt.

Bây giờ thêm nhiều. Bạn đo hướng không chính xác tuyệt đối, nên các tia không giao nhau ở đúng một điểm — chúng tạo thành một chùm tia hơi lệch nhau. Không có vị trí nào thoả mãn *tất cả* các phép đo. Câu hỏi đổi thành: vị trí nào làm tổng bình phương độ lệch nhỏ nhất?

Và đây là chỗ hai tầng xuất hiện. Lời giải dạng đóng giống như việc dùng một công thức sẵn có để ra ngay một đáp án gần đúng — nhanh, và nó tối thiểu hoá một thứ gì đó tiện tính chứ không phải thứ bạn thật sự quan tâm. Tình hình giống như việc đứng ở đáp án ấy rồi dịch chuyển từng chút một theo hướng làm tổng lệch giảm, cho tới khi không giảm được nữa.

Điều đáng nói cuối: khi bạn dừng lại, độ *cong* của mặt tổng-lệch quanh chỗ bạn đứng cho bạn biết bạn chắc chắn tới mức nào. Dốc đứng theo mọi hướng thì vị trí xác định tốt; có một hướng gần phẳng thì theo hướng ấy bạn không biết gì cả. Đó chính là ma trận thông tin của $A/05$, và nó chỉ tồn tại nếu bạn đã làm bước tinh chỉnh.

1.2 P3P: ba điểm và bốn nghiệm

Trường hợp tối thiểu, và nó đáng hiểu vì nó là hạt nhân của RANSAC.

Cho ba điểm P_1, P_2, P_3 đã biết khoảng cách đôi một d_{12}, d_{13}, d_{23} , và ba hướng tia đơn vị $\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3$ từ tâm camera tới chúng (tính được từ toạ độ ảnh và K^{-1}). Ẩn là ba khoảng cách s_1, s_2, s_3 từ tâm camera tới ba điểm.

Định lý cosin trong ba tam giác cho ba phương trình:

$$s_i^2 + s_j^2 - 2s_i s_j \cos \theta_{ij} = d_{ij}^2, \quad \cos \theta_{ij} = \mathbf{v}_i^\top \mathbf{v}_j.$$

Ba phương trình bậc hai, ba ẩn. Khử hai ẩn cho ra một đa thức bậc bốn theo ẩn còn lại, và đó là nguồn gốc của con số bốn trong “tối đa bốn nghiệm” — đây là câu trả lời cho Q1. Không phải hai vì hệ thực sự bậc bốn; không phải tám vì các nghiệm âm (điểm sau camera) bị loại bởi vật lý.

Sau khi có s_i , toạ độ ba điểm trong hệ camera là $s_i \mathbf{v}_i$, và pose thu được bằng cách khớp hai tập ba điểm — một bài toán Procrustes giải bằng SVD.

Ý nghĩa thực hành của “bốn nghiệm” là rất cụ thể: *bạn phải có điểm thứ tư để chọn*. Quy trình chuẩn là dùng ba điểm giải P3P, được tới bốn ứng viên, rồi chiếu điểm thứ tư bằng từng ứng viên và giữ ứng viên cho sai số tái chiếu nhỏ nhất. Đây là lý do mọi cài đặt RANSAC cho PnP dùng tập con bốn điểm chứ không phải ba.

Có nhiều công thức hoá khác nhau cho P3P, khác nhau ở độ ổn định số học và tốc độ; công thức của Kneip và cộng sự tránh được bước khớp Procrustes bằng cách xây trực tiếp một hệ toạ độ trung gian [3]. Với mục đích của cây thì chi tiết ấy không quan trọng — điều quan trọng là biết P3P tồn tại, nó cho nhiều nghiệm, và vai trò của nó là *hạt nhân tối thiểu cho RANSAC*, không phải bộ giải chính.

CẠM BÃY

Đừng nhầm bốn nghiệm của P3P với hai nghiệm của target phẳng ở A/04. Chúng là hai hiện tượng khác nhau và cách xử lý khác nhau.

Bốn nghiệm của P3P là hệ quả của việc *thiếu dữ liệu*: ba điểm không đủ để xác định sáu bậc tự do một cách duy nhất. Thêm một điểm thứ tư (không suy biến) là loại được ba trong bốn, và vấn đề biến mất hoàn toàn.

Hai nghiệm của target phẳng là hệ quả của một *đối xứng hình học* và nó *không* biến mất khi thêm điểm — miễn là các điểm thêm vào vẫn đồng phẳng. Đó là một loại bệnh hoàn toàn khác, chữa bằng cách phá tính đồng phẳng chứ không bằng cách thêm dữ liệu.

Hệ quả cài đặt: một tag vuông đơn lẻ có bốn góc, đủ để loại bớt nghiệm P3P, nhưng *vẫn* có hai nghiệm phẳng. Nếu bạn thấy hai nghiệm sau khi đã dùng cả bốn góc thì đó không phải bug.

1.3 Các lời giải dạng đóng cho n điểm

Bốn họ, mỗi họ một ý tưởng.

DLT. Viết phương trình chiếu thành hệ tuyến tính theo mười hai phần tử của $[R|t]$, bỏ qua ràng buộc trực giao, giải bằng SVD, rồi chiếu về $SO(3)$ sau. Đơn giản nhất, và cũng kém chính xác nhất, vì nó “quên” sáu ràng buộc rồi sửa sau. Nó cũng không dùng được K đã biết một cách hiệu quả. Vai trò thực tế: giá trị giáo dục, và khởi tạo khi không có gì tốt hơn.

EPnP. Ý tưởng đẹp: biểu diễn n điểm 3D bằng tổ hợp affine của bốn “điểm điều khiển ảo”, rồi giải cho toạ độ camera của bốn điểm ảo ấy thay vì cho n điểm. Bài toán rút gọn về kích thước cố định bất kể n , nên độ phức tạp là $O(n)$ [4]. Đây là lựa chọn tốt cho bài toán có nhiều điểm.

IPPE. Chuyên cho target phẳng. Nó khai thác cấu trúc phẳng bằng cách phân tích phép biến đổi *vi phân* quanh mỗi điểm, và điều quan trọng nhất với ta: nó trả về *cả hai* nghiệm cùng sai số tái chiếu của từng nghiệm [5]. Với marker vuông — tức gần như toàn bộ bài toán của cây — đây là lựa chọn đúng, và biến thể cho tag vuông có trong OpenCV dưới tên SOLVEPNP_IPPE_SQUARE.

SQPnP và các phương pháp tối ưu toàn cục. Giải bài toán bình phương tối thiểu trên $SO(3)$ bằng lập trình bậc hai tuần tự, đảm bảo tìm được cực tiểu toàn cục của một hàm mục tiêu đại số [6]. Chúng khắc phục nhược điểm “không tối ưu” của các phương pháp trước ở mức đại số, nhưng vẫn không phải nghiệm tối ưu theo sai số tái chiếu.

Bảng 1.1. Bốn họ lời giải dạng đóng: ý tưởng, ưu nhược, và vai trò trong pipeline marker.

Phương pháp	Ý tưởng	Điểm yếu	Vai trò cho marker
P3P	Định lý cosin, đa thức bậc bốn	Tối đa 4 nghiệm, cần điểm thứ tư để chọn	Hạt nhân RANSAC; không dùng làm bộ giải chính
DLT	Bỏ ràng buộc trực giao, giải tuyến tính	Kém chính xác nhất; không tận dụng K	Khởi tạo dự phòng; giá trị giáo dục
EPnP [4]	Bốn điểm điều khiển ảo, $O(n)$	Cực tiểu sai số đại số, không phải tái chiếu	Tốt khi nhiều điểm và không đồng phẳng
IPPE [5]	Khai thác cấu trúc phẳng, phân tích vi phân	Chỉ cho target phẳng	Lựa chọn mặc định cho marker vuông — cho cả hai nghiệm

Ghi chú trung thực: tôi không đưa vào chương này các con số so sánh tốc độ hay độ chính xác giữa các thuật toán, dù chúng có sẵn trong các bài báo gốc. Lý do là những con số ấy được đo trên các cấu hình rất khác nhau — số điểm, mức nhiễu, phân bố hình học — và trích chúng ra khỏi ngữ cảnh thì chúng không so sánh được với nhau. Điều tôi khẳng định ở đây chỉ là các *tính chất cấu trúc*: IPPE cho cả hai nghiệm của target phẳng, EPnP là $O(n)$, P3P cho tối đa bốn nghiệm. Ba điều ấy là tính chất toán học chứ không phải kết quả đo.

1.4 Tinh chỉnh: nghiệm thật sự

Đây là bước quyết định, và nó là bước cấp Jacobian cho A/05.

Hàm mục tiêu là sai số tái chiếu có trọng số:

$$C(\xi) = \frac{1}{2} \sum_{i=1}^N \mathbf{r}_i(\xi)^\top \Sigma_i^{-1} \mathbf{r}_i(\xi), \quad \mathbf{r}_i(\xi) = u_i - \hat{u}_i(\xi). \quad (1.1)$$

Đây là hàm hợp lý âm (negative log-likelihood) dưới giả thiết nhiễu Gauss độc lập trên tọa độ pixel, nên nghiệm của nó là ước lượng hợp lý cực đại — đó là lý do nó đáng làm, chứ không phải vì nó “chính xác hơn” một cách mơ hồ.

A. Tham số hoá xoay: vì sao phải dùng $\mathfrak{se}(3)$

Trước khi lấy đạo hàm, phải quyết định biểu diễn ξ thế nào. Đây là chỗ Q4 chỉ tới và nó là một lỗi cài đặt hay gặp.

Vấn đề: R có chín phần tử nhưng chỉ ba bậc tự do. Tối ưu trực tiếp trên chín phần tử với sáu ràng buộc là phiền; tối ưu trên góc Euler thì gặp **khóa gimbal** (*gimbal lock*) — ở một số tư thế, hai trong ba trục trùng nhau và Jacobian mất hạng. Triệu chứng đúng như Q4 mô tả: chạy tốt phần lớn thời gian rồi phân kỳ ở một số tư thế cụ thể, lặp đi lặp lại ở đúng những tư thế ấy.

Cách đúng là dùng một *tham số hoá cục bộ*: giữ ước lượng hiện tại R_k dưới dạng ma trận (hoặc quaternion), và tham số hoá *số gia* bằng một vectơ ba chiều ω qua ánh xạ mũ:

$$R_{k+1} = R_k \exp(\omega^\wedge),$$

với $\cdot^\wedge$ là toán tử đưa vectơ ba chiều thành ma trận phản đối xứng. Tại mỗi bước lặp ta tối ưu trên ω quanh không, nơi tham số hoá không có kỳ dị, rồi cập nhật và đặt lại. Đây là ý tưởng chuẩn của tối ưu trên đa tạp, và một tài liệu tham chiếu gọn cho nó là [7].

B. Bước Gauss–Newton

Tuyến tính hoá $\mathbf{r}_i$ quanh ξ_k :

$$\mathbf{r}_i(\xi_k + \delta) \approx \mathbf{r}_i(\xi_k) - J_i \delta, \quad J_i = \left. \frac{\partial \hat{u}_i}{\partial \xi} \right|_{\xi_k}.$$

Thay vào (1.1), lấy đạo hàm theo δ và cho bằng không:

$$\left(\sum_i J_i^\top \Sigma_i^{-1} J_i \right) \delta = \sum_i J_i^\top \Sigma_i^{-1} \mathbf{r}_i. \quad (1.2)$$

Đây là *phương trình chuẩn*, và ma trận bên trái là chính xác ma trận thông tin $\mathcal{I}$ mà A/05 đảo để ra hiệp phương sai. Điểm đáng nhấn mạnh: $\mathcal{I}$ không phải một thứ phải tính thêm — nó *đã có sẵn* như một sản phẩm phụ của mỗi bước lặp. Bỏ qua nó là vứt đi thông tin miễn phí.

Với sáu ẩn, hệ (1.2) là 6×6 — giải trong micro giây. Đây cũng là lý do PnP tinh chỉnh rẻ đến mức không có lý do gì để bỏ qua: vài chục bước lặp trên một hệ 6×6 là không đáng kể so với chi phí chạy detector.

LM thêm số hạng $\lambda \text{diag}(\mathcal{I})$ vào về trái, điều chỉnh λ theo mức độ thành công của từng bước. Nó bền hơn khi khởi tạo xa hoặc khi $\mathcal{I}$ kém điều kiện — tức đúng các tình huống mà A/02 mục 4 cảnh báo.

C. Jacobian của phép chiếu

Để hoàn chỉnh, đây là mảnh ghép cuối. Với điểm trong hệ camera $\mathbf{X}_c = (X, Y, Z)$, phép chiếu là $\hat{u} = fX/Z + c_x$, $\hat{v} = fY/Z + c_y$. Đạo hàm theo $\mathbf{X}_c$:

$$\frac{\partial \hat{u}}{\partial \mathbf{X}_c} = \frac{f}{Z} \begin{pmatrix} 1 & 0 & -X/Z \\ 0 & 1 & -Y/Z \end{pmatrix}.$$

Nhìn ma trận này một lát thì thấy ngay nguồn gốc của mọi kết luận ở A/05: hệ số chung là f/Z , giải thích vì sao sai số ngang là $Z\sigma/f$; và cột thứ ba — đạo hàm theo chiều sâu — mang thêm hệ số X/Z và Y/Z , tức *tỉ lệ với độ lệch khỏi trục quang*. Một điểm nằm đúng trên trục quang có $X = Y = 0$ nên đạo hàm theo Z bằng không: dịch nó ra xa hay lại gần không đổi hình chiếu chút nào. Đây chính là cơ chế sâu xa vì sao chiều sâu khó đo, và vì sao nó chỉ đo được nhờ các điểm *cách xa trục quang* — tức nhờ kích thước tag.

Phần còn lại của Jacobian là đạo hàm của $\mathbf{X}_c$ theo ξ , và nó có dạng chuẩn $[I \quad -\mathbf{X}_c^\wedge]$ với tham số hoá $\mathfrak{se}(3)$. Ghép lại bằng quy tắc chuỗi.

CẦN NHỚ

Ba lý do bắt buộc phải tinh chỉnh, không phải một.

Một, độ chính xác: lời giải dạng đóng cực tiểu sai số đại số, tinh chỉnh cực tiểu sai số tái chiếu — chỉ cái sau là ước lượng hợp lý cực đại.

Hai, và đây là lý do hay bị quên: tinh chỉnh sinh ra ma trận thông tin $\mathcal{I} = \sum J^\top \Sigma^{-1} J$ miễn phí. Không có nó thì không có hiệp phương sai để hợp nhất, không có cơ sở loại ngoại lai theo chi-square, và không có gì đưa vào ngân sách sai số.

Ba, chi phí gần như bằng không: một hệ 6×6 giải vài chục lần, không đáng kể so với detector.

Kết luận thực hành: solvePnPRefineLM hoặc tương đương phải luôn có trong pipeline, ngay sau lời giải dạng đóng.

1.5 PnP hợp nhất trên nhiều tag

Đây là mục quan trọng nhất của chương với bài toán docking, và nó trả lời Q3.

Tình huống: M tag, mỗi tag bốn góc, tọa độ 3D của tất cả $4M$ góc đã biết *trong cùng một hệ tọa độ dock*. Câu hỏi: tìm pose của camera so với hệ dock.

Cách đúng là chạy (1.1) với $N = 4M$, tức một bài toán tối ưu duy nhất trên toàn bộ tập góc. Không có gì đặc biệt về mặt thuật toán — vẫn là (1.2) với tổng chạy qua nhiều số hạng hơn. Cái đặc biệt nằm ở chỗ *thông tin hình học được sử dụng*.

Để thấy rõ, so sánh hai cách bằng ngôn ngữ của ma trận thông tin.

Cách hợp nhất cộng các ma trận thông tin trong cùng một hệ tọa độ:

$$\mathcal{I}_{\text{hợp nhất}} = \sum_{m=1}^M \sum_{i \in \text{tag } m} J_i^T \Sigma_i^{-1} J_i.$$

Các J_i ở đây được tính với tọa độ 3D thật của từng góc trong hệ dock, nên chúng *biết* rằng tag 1 nằm cách tag 2 một khoảng B .

Cách trung bình pose tính M pose riêng, mỗi pose có hiệp phương sai $\Sigma_m = \mathcal{I}_m^{-1}$ tính trong hệ của riêng tag ấy, rồi gộp. Ngay cả khi gộp đúng cách (trung bình có trọng số theo nghịch đảo hiệp phương sai, trên đa tạp), kết quả vẫn khác, vì phép nghịch đảo và phép tổng không giao hoán:

$$\left(\sum_m \mathcal{I}_m \right)^{-1} \neq \left(\sum_m \mathcal{I}_m^{-1} \right)^{-1} \dots$$

và quan trọng hơn cả bất đẳng thức đại số: khi tính $\mathcal{I}_m$ riêng cho từng tag, ta đã *vứt bỏ* ràng buộc rằng các tag gắn cứng với nhau. Ràng buộc ấy chính là thứ tạo ra cơ chế baseline của A/05 mục 6.

Lưu ý điểm này là trường hợp riêng của một nguyên lý tổng quát trong hiệu chỉnh chùm tia: *cộng thông tin, đừng cộng ước lượng* [8, §2]. Nó xuất hiện ở khắp nơi trong ước lượng trạng thái, và nó luôn nói cùng một điều: gộp ở tầng phần dư thì giữ được cấu trúc, gộp ở tầng kết quả thì mất.

CẠM BẮY

Có một điều kiện tiên quyết cho PnP hợp nhất mà nếu không thoả thì nó *tệ hơn* trung bình pose: *tọa độ 3D của các tag trong hệ dock phải đúng*.

Lý do: PnP hợp nhất tin vào mô hình constellation và ép một pose duy nhất giải thích mọi góc. Nếu mô hình sai — tag 2 thực ra cách tag 1 là 398 mm chứ không phải 400 mm như CAD — thì bộ giải phải thoả hiệp, và nó phân bổ sai lệch ấy vào pose theo một cách không kiểm soát được. Trung bình pose thì “rộng lượng” hơn: mỗi pose riêng vẫn đúng trong hệ của tag nó, và sai lệch constellation chỉ làm các pose tản mát.

Hệ quả: *PnP hợp nhất chỉ tốt hơn nếu bạn đã đo constellation cẩn thận*, và đó là lý do B/12 đặt bước hiệu chuẩn constellation bằng bundle adjustment ở mức bắt buộc chứ không phải tùy chọn.

May mắn là có một chỉ báo: nếu sai số tái chiếu sau khi hợp nhất lớn hơn hẳn so với khi giải từng tag riêng, thì mô hình constellation của bạn sai. Đây là một phép thử miễn phí và nó đáng chạy định kỳ.

1.6 Trọng số: không phải góc nào cũng như nhau

Công thức (1.1) có Σ_i cho từng góc, và phần lớn cài đặt đặt $\Sigma_i = \sigma^2 I$ như nhau cho mọi góc. Mục này trả lời Q5: giả định ấy sai theo chiều nào, và sửa thế nào.

Bốn nguồn làm Σ_i khác nhau giữa các góc.

Khoảng cách và kích thước biểu kiến. Một tag ở xa có kích thước nhỏ trên ảnh; các cạnh của nó ngắn hơn nên đường thẳng khớp lên chúng kém xác định hơn, nên góc kém chính xác hơn. Đây là nguồn lớn nhất trong bài toán docking vì các tag trong constellation có thể ở các khoảng cách rất khác nhau khi robot tới gần.

Góc nghiêng. Một tag nhìn rất chếch có các cạnh bị nén, và số pixel dọc theo cạnh ít đi, nên định vị kém hơn. Thêm nữa, cạnh bị nén có gradient thay đổi theo cách khác, làm phép khớp đường kém ổn định.

Vị trí trên ảnh. Méo còn sót lại lớn hơn ở rìa ($A/01$), và nó thể hiện như một thành phần thiên lệch — không phải nhiễu, nhưng nếu ta không mô hình hoá được nó thì cách thực dụng là tăng Σ cho các góc ở rìa để giảm ảnh hưởng của chúng.

Điều kiện quang học cục bộ. Một góc nằm trong vùng bị loá, hoặc bị mờ do độ sâu trường ảnh, hoặc có tương phản thấp, thì kém chính xác hơn ($A/06$, $B/14$).

Hai cách hiện thực hoá. Cách đơn giản là gán Σ_i theo một quy tắc heuristic — chẳng hạn tỉ lệ nghịch với kích thước tag trên ảnh — và điều đó đã tốt hơn nhiều so với trọng số đều. Cách đúng hơn là để chính detector ước lượng bất định của mỗi góc: bước khớp đường thẳng ở $B/08$ sinh ra một hiệp phương sai cho mỗi góc như sản phẩm phụ, và nó mang cả thông tin về *hướng* — một góc trên một cạnh dài và sắc nét được xác định tốt theo hướng vuông góc cạnh và kém hơn theo hướng dọc cạnh, tức Σ_i dị hướng.

Khung xử lý dị hướng đúng cách đã được đưa ra trong MLPnP [9], và ý tưởng của nó áp dụng thẳng vào đây: dùng hiệp phương sai thật của từng phép đo thay vì một hằng số. Lợi ích không chỉ là độ chính xác mà còn là *tính trung thực của hiệp phương sai đầu ra* — nếu Σ_i đầu vào đúng thì T^{-1} đầu ra mới là một ước lượng bất định đáng tin, và cả chuỗi hợp nhất ở $B/11$ phụ thuộc vào điều đó.

1.7 Loại ngoại lai

Bình phương tối thiểu rất nhạy với điểm ngoại lai: một góc sai 50 px đóng góp vào hàm mục tiêu gấp 250^2 lần một góc sai 0,2 px, nên nó kéo nghiệm về phía nó.

Trong bài toán marker, ngoại lai đến từ đâu? Không nhiều như trong bài toán đặc trưng tự nhiên — đó là một lợi thế lớn của marker. Nhưng vẫn có: một tag bị che một phần cho góc sai; một tag bị bắn hoặc mòn cho decode đúng nhưng góc lệch; một tag decode nhầm hướng cho bốn tương ứng xoay 90 độ; và một tag trong constellation bị lắp sai vị trí so với mô hình cho một sai lệch có hệ thống trông như ngoại lai.

Ba lớp công cụ, dùng theo thứ tự.

RANSAC ở mức tag, không ở mức góc. Vì bốn góc của một tag hoặc cùng đúng hoặc cùng sai, việc lấy mẫu ngẫu nhiên ở mức góc là lãng phí. Lấy mẫu ở mức tag: chọn ngẫu nhiên hai tag, giải PnP, đếm số tag phù hợp, lặp [1]. Rẻ hơn và đúng cấu trúc hơn.

Cổng chi-square trên phần dư. Sau khi hội tụ, phần dư chuẩn hoá $\mathbf{r}_i^T \Sigma_i^{-1} \mathbf{r}_i$ theo lý thuyết có phân bố χ^2 hai bậc tự do. Loại các góc vượt quá một phân vị — chẳng hạn 99% — rồi giải lại. Đây là phép thử có cơ sở thống kê chứ không phải một ngưỡng tùy ý, và nó *đòi* Σ_i đúng — một lý do nữa để làm mục 6 cho tử tế.

Hàm mất mát bền (robust loss). Thay bình phương bằng Huber hoặc Cauchy, làm giảm ảnh hưởng của phần dư lớn mà không loại hẳn. Ưu điểm so với loại cứng: liên tục, nên không gây nhảy khi một điểm dao động quanh ngưỡng. Nhược điểm: nó làm hiệp phương sai đầu ra khó diễn giải hơn.

CẦN NHỚ

Có một phép kiểm nhất quán riêng cho constellation mà không hệ nào tự làm và nó rất đáng thêm: *giải PnP riêng cho từng tag, rồi so các pose với nhau.*

Nếu constellation đúng và mọi tag lành, các pose ấy phải nhất quán trong phạm vi hiệp phương sai của chúng. Nếu pose từ tag A và tag B lệch nhau quá ngưỡng, thì một trong ba điều đã sai: mô hình constellation sai (tag bị lắp lệch, hoặc CAD sai), một tag bị che hoặc bắn, hoặc một tag đang ở nghiệm ambiguity khác.

Phép kiểm này không thay thế PnP hợp nhất — nó chạy *song song* với nó như một giám sát. Chi phí thấp, và nó bắt được đúng loại lỗi mà sai số tái chiếu của bài toán hợp nhất có thể che giấu khi số tag lớn.

1.8 Giới hạn và chế độ hỏng

Giả thiết K và hệ số méo đúng. PnP coi chúng là hằng số đã biết. Nếu chúng sai thì PnP vẫn hội tụ, sai số tái chiếu vẫn nhỏ, và pose sai — không có tín hiệu nào. Đây là lý do A/01 nằm trước chương này và B/12 nằm ở mức xương sống.

Giả thiết tọa độ 3D đúng. Như mục 5 đã nêu, PnP hợp nhất khuếch đại hậu quả của sai số constellation. Chỉ báo: so sai số tái chiếu giữa hợp nhất và từng tag.

Giả thiết tương ứng đúng. PnP giả định biết góc nào ứng với điểm 3D nào. Với tag có mã thì decode giải quyết, nhưng decode sai hướng cho một chế độ hỏng thâm lặn: pose sai đúng 90 độ và sai số tái chiếu vẫn nhỏ vì hình vuông đối xứng bốn lần. Phép bắt: kiểm tính nhất quán với tag khác trong constellation.

Tối ưu cục bộ. Tinh chỉnh hội tụ về cực tiểu gần nhất, không phải cực tiểu toàn cục. Với target phẳng thì có đúng hai cực tiểu (A/04), và khởi tạo quyết định bạn rơi vào cái nào. Đây là lý do khởi tạo bằng IPPE — vốn tìm cả hai — tốt hơn khởi tạo bằng bất cứ thứ gì khác.

Giả thiết nhiễu Gauss. Hàm mục tiêu bình phương là hợp lý cực đại chỉ dưới giả thiết này. Với nhiễu có đuôi nặng (ngoại lai), nó không còn tối ưu — đó là lý do mục 7 tồn tại.

1.9 Thực hành

Pipeline khuyến nghị, năm bước, và nó là công thức tôi khuyên dùng làm mặc định.

Một, khử méo tọa độ góc (hoặc đưa méo vào hàm chiếu — A/01 mục 7).

Hai, khởi tạo bằng IPPE cho target phẳng, lấy cả hai nghiệm và tỉ số sai số tái chiếu ρ của chúng. Với constellation không đồng phẳng, dùng EPnP hoặc SQPnP.

Ba, tinh chỉnh LM trên toàn bộ tập góc của mọi tag nhìn thấy, với tọa độ 3D trong hệ dock và trọng số theo Σ_i từ mục 6.

Bốn, công chi-square trên phần dư, loại góc ngoại lai, giải lại nếu có loại.

Năm, xuất ra ba thứ chứ không phải một: pose, hiệp phương sai $\mathcal{I}^{-1}$, và ρ . Ba con số ấy là tối thiểu để tăng trên (B/11, B/13) làm việc có cơ sở.

Trong OpenCV, chuỗi ấy là `undistortPoints` $\rightarrow$ `solvePnPGeneric` với `SOLVEPNP_IPPE_SQUARE` $\rightarrow$ `solvePnPRefineLM`, cộng phần tính $\mathcal{I}$ tự viết (OpenCV không trả về nó, và đây là một thiếu sót đáng tiếc của API).

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● Hai tầng của PnP	Dạng đóng (P3P, EPnP, IPPE, DLT) cực tiểu sai số <i>đại số</i> , không cần khởi tạo, nhanh. Tinh chỉnh phi tuyến cực tiểu sai số <i>tái chiếu</i> — đây mới là ước lượng hợp lý cực đại và là nghiệm cuối.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Vì sao P3P có tối đa 4 nghiệm	Khử hai trong ba phương trình định lý cosin cho một đa thức bậc bốn [2]. Cần điểm thứ tư để chọn — đó là lý do RANSAC cho PnP dùng tập con 4 điểm.
●●● 4 nghiệm P3P vs 2 nghiệm phẳng	Khác loại. 4 nghiệm P3P là do <i>thiếu dữ liệu</i> , biến mất khi thêm điểm. 2 nghiệm phẳng là do <i>đối xứng hình học</i> , không biến mất khi thêm điểm đồng phẳng (A/04).
●●● Ba lý do phải tinh chỉnh	Độ chính xác (hợp lý cực đại); ma trận thông tin $\mathcal{I} = \sum J^T \Sigma^{-1} J$ sinh ra <i>miễn phí</i> ; và chi phí gần bằng không (hệ 6×6).
●●● Phương trình chuẩn	$(\sum_i J_i^T \Sigma_i^{-1} J_i) \delta = \sum_i J_i^T \Sigma_i^{-1} \mathbf{r}_i$. Ma trận về trái chính là $\mathcal{I}$ mà A/05 đảo.
●●● Vì sao không dùng góc Euler	Khoá gimbal: ở một số tư thế hai trục trùng nhau, Jacobian mất hạng, bộ giải phân kỳ. Triệu chứng đặc trưng: hỏng lặp lại ở đúng vài tư thế cụ thể. Cách đúng: tham số hoá cục bộ $R_{k+1} = R_k \exp(\omega^\wedge)$ [7].
●●● Jacobian của phép chiếu	$\partial \mathbf{u} / \partial \mathbf{X}_c = (f/Z) \begin{pmatrix} 1 & 0 & -X/Z \\ 0 & 1 & -Y/Z \end{pmatrix}$. Hệ số chung f/Z giải thích $\delta x = Z \sigma / f$; cột thứ ba tỉ lệ với X/Z — điểm trên trục quang không cho thông tin chiều sâu nào.
●●● PnP hợp nhất vs trung bình pose	Hợp nhất cộng <i>thông tin</i> trong cùng hệ toạ độ, nên giữ được ràng buộc rằng các tag gắn cứng với nhau — tức giữ được cơ chế baseline. Trung bình pose cộng <i>ước lượng</i> , vứt bỏ ràng buộc ấy [8, §2].
●●● Điều kiện để hợp nhất tốt hơn	Toạ độ 3D constellation phải đúng. Nếu sai, hợp nhất <i>tệ hơn</i> trung bình vì nó ép một pose duy nhất giải thích mô hình sai. Chỉ báo: số sai số tái chiếu hợp nhất với từng tag riêng.
●● Bốn nguồn làm Σ_i khác nhau	Khoảng cách và kích thước biểu kiến (lớn nhất trong docking); góc nghiêng; vị trí trên ảnh (méo còn sót ở rìa); điều kiện quang học cục bộ.
●● Vì sao trọng số đúng quan trọng hơn độ chính xác	Vì $\mathcal{I}^{-1}$ chỉ là ước lượng bất định đáng tin nếu Σ_i đầu vào đúng, và cả chuỗi hợp nhất ở B/11 lẫn công chi-square ở mục 7 phụ thuộc vào tính đúng ấy [9].
●● RANSAC ở mức nào	Mức <i>tag</i> , không mức góc — vì bốn góc một tag cùng đúng hoặc cùng sai. Chọn ngẫu nhiên hai tag, giải, đếm tag phù hợp.
●● Công chi-square	Phần dư chuẩn hoá $\mathbf{r}^T \Sigma^{-1} \mathbf{r} \sim \chi^2_2$; loại góc vượt phân vị 99% rồi giải lại. Có cơ sở thống kê, nhưng <i>đòi</i> Σ đúng.
●● Phép kiểm nhất quán constellation	Giải PnP riêng từng tag rồi so các pose. Lệch quá ngưỡng $\Rightarrow$ constellation sai, tag bị che/bẩn, hoặc một tag đang ở nghiệm ambiguity khác. Chạy song song với hợp nhất như một giám sát.
● Chế độ hỏng thâm lặng nhất	Decode sai hướng: pose sai đúng 90° và sai số tái chiếu <i>vẫn nhỏ</i> vì hình vuông đối xứng bốn lần. Chỉ bắt được bằng kiểm nhất quán với tag khác.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
OpenCV solvePnPGeneric	Trả về nhiều nghiệm kèm sai số tái chiếu từng nghiệm; hỗ trợ IPPE, EPnP, SQPnP, P3P	API quan trọng nhất của cả cây — xem D/17
OpenCV solvePnPRefineLM	Bước tinh chỉnh LM trên sai số tái chiếu	Bắt buộc có trong pipeline; rẻ
apriltag apriltag_pose.c	Cài IPPE cho tag vuông, tính cả hai nghiệm và tỉ số	Đọc để thấy mục 3 và A/04 thành code

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
Ceres [10], GTSAM	Tối ưu tổng quát với tham số hoá trên đa tạp và hàm mất mát bền	Dùng khi bài toán vượt quá một pose duy nhất — B/12
MLPnP [9]	PnP với hiệp phương sai dị hướng cho từng phép đo	Nguồn cho mục 6; ý tưởng áp thẳng vào trọng số góc marker

MINI PROJECT · So PnP hợp nhất với trung bình pose, và đo cái giá của constellation sai · 2 ngày

Mục tiêu	chuyển luận điểm trung tâm của mục 5 từ một lập luận thành một cặp đường cong, và đồng thời đo được ngưỡng sai số constellation mà trên đó hợp nhất bắt đầu tệ hơn.
Dữ liệu	mô phỏng, nối tiếp script của A/05 và A/04.
Các bước	dựng constellation bốn tag với baseline B , tọa độ 3D chân trị đã biết; chiếu, thêm nhiễu σ ; chạy hai đường: (i) PnP hợp nhất trên 16 góc, (ii) bốn PnP riêng rồi trung bình Karcher trên $SO(3)$ cộng trung bình có trọng số cho tịnh tiến; so sai số góc và sai số vị trí của hai cách qua 2000 lần chạy; lặp khi quét B . Sau đó phần thứ hai: cố ý làm sai mô hình constellation — dịch tọa độ 3D của một tag đi e mm so với chân trị — và quét e từ 0 tới 5 mm, xem cách nào xuống dốc nhanh hơn.
Xong khi	bạn có hai đồ thị. Đồ thị một cho thấy hợp nhất thẳng rõ về sai số góc và khoảng cách thẳng tăng theo B . Đồ thị hai cho thấy một giao điểm: với e vượt một ngưỡng nào đó, trung bình pose bắt đầu tệ hơn. Con số ngưỡng ấy chính là yêu cầu độ chính xác cơ khí cho constellation của bạn. Kiểm tra chéo bắt buộc: ở $e = 0$ và B rất nhỏ, hai cách phải cho kết quả gần như nhau — nếu không thì phép trung bình Karcher của bạn có lỗi.
Nối với	B/09 và B/12 nhận con số ngưỡng này làm yêu cầu thiết kế; C/15 đưa nó vào bảng ngân sách.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Bốn vì hệ ba phương trình định lý cosin, sau khi khử hai ẩn, cho một đa thức bậc bốn theo ẩn còn lại [2]; không phải hai vì hệ thực sự bậc bốn chứ không suy biến, và không phải tám vì các nghiệm ứng với điểm nằm sau camera bị vật lý loại. Ý nghĩa ở tầng code rất cụ thể: bạn *phải* có một điểm thứ tư để chọn trong bốn ứng viên, bằng cách chiếu nó qua từng ứng viên và giữ cái cho sai số tái chiếu nhỏ nhất. Đó là lý do mọi cài đặt RANSAC cho PnP lấy mẫu bốn điểm chứ không ba, dù ba mới là số tối thiểu để giải.
- A2. Vì EPnP cực tiểu hoá một sai số *đại số* — độ lệch trong hệ phương trình tuyến tính của bốn điểm điều khiển ảo — chứ không phải sai số tái chiếu tính bằng pixel [4]. Hai đại lượng ấy không tỉ lệ với nhau, nên nghiệm tối ưu theo cái này không tối ưu theo cái kia, và chênh lệch đáng kể khi nhiễu không nhỏ. Nhưng có một lý do thứ hai quan trọng không kém: EPnP không sinh ra ma trận thông tin. Không có $\mathcal{I}$ thì không có hiệp phương sai để hợp nhất ở B/11, không có cơ sở cho cổng chi-square, và không có gì đưa vào ngân sách sai số. Bước tinh chỉnh cho cả hai thứ với chi phí gần bằng không.
- A3. Khác nhau vì PnP hợp nhất *biết* rằng bốn tag gắn cứng với nhau ở những vị trí đã biết, còn trung bình pose thì không. Cụ thể: hợp nhất cộng các ma trận thông tin $J_i^T \Sigma_i^{-1} J_i$ trong cùng một hệ tọa độ, với J_i tính từ tọa độ 3D thật của từng góc trong hệ dock — nên thông tin về hướng đến từ chênh lệch vị trí giữa các tag cách nhau B , tức cơ chế baseline của A/05. Trung bình pose tính hiệp phương sai riêng cho từng tag trong hệ của tag ấy, vớt bỏ ràng buộc liên tag, nên mỗi pose vẫn chịu hệ số $\sin \theta$ và trung bình chỉ giảm nhiễu theo $\sqrt{M}$. Cách đúng là hợp nhất — với một điều kiện: tọa độ 3D constellation phải đúng, nếu không thì hợp nhất lại tệ hơn.
- A4. Khoá gimbal. Góc Euler có kỳ dị ở một số tư thế nơi hai trục quay trùng nhau; tại đó ánh xạ từ ba tham số sang $SO(3)$ mất hạng, nên Jacobian mất hạng, nên phương trình chuẩn suy biến và bước cập nhật nổ tung. Triệu chứng đặc trưng và chẩn đoán được: nó hỏng *lặp lại ở đúng vài tư thế cụ thể* chứ không ngẫu nhiên — nếu bạn thấy phân kỳ luôn xảy ra quanh cùng một giá trị pitch thì gần như chắc chắn là chuyện này. Cách sửa đúng không phải thêm chính quy hoá

hay đổi ngưỡng mà là đổi tham số hoá: giữ R dưới dạng ma trận hoặc quaternion, tham số hoá số *gia* bằng vectơ ba chiều qua ánh xạ mũ, và đặt lại sau mỗi bước — tức tối ưu quanh gốc của $se(3)$ nơi không có kỳ dị [7].

- A5. Bạn đang giả định rằng mọi góc có cùng độ tin cậy, và giả định ấy sai theo chiều *nguy hiểm*: các góc kém nhất được cho cùng tiếng nói với các góc tốt nhất, nên chúng kéo nghiệm về phía chúng nhiều hơn mức đáng có. Cụ thể trong tình huống mô tả: tag ở xa có kích thước nhỏ trên ảnh nên cạnh ngắn nên định vị góc kém, và tag nghiêng mạnh có cạnh bị nén nên càng kém — cả hai hiệu ứng cộng lại có thể làm σ của nó lớn gấp nhiều lần tag gần. Hậu quả thứ hai, và nó tệ hơn hậu quả thứ nhất: hiệp phương sai $\mathcal{I}^{-1}$ mà bạn xuất ra sẽ *sai*, vì nó được tính từ Σ_i giả định. Nó sẽ báo một bất định nhỏ hơn thực tế, và mọi tầng phía trên dùng con số ấy làm trọng số sẽ bị lừa theo.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn đứng giữa một cánh đồng và nhìn thấy vài cái cột mà bạn biết chính xác chúng ở đâu. Bạn không đo được khoảng cách tới chúng, chỉ đo được hướng: cột này ở bên trái một chút, cột kia ở bên phải nhiều hơn. Câu hỏi là bạn đang đứng ở đâu và quay mặt về hướng nào.

Một cái cột thì chưa nói được gì. Ba cái thì gần đủ — nhưng chỉ gần, vì hoá ra vẫn còn tới bốn chỗ mà từ đó ba cái cột trông đúng như vậy. Phải có cái cột thứ tư mới loại được ba chỗ giả.

Với phép đo hoàn hảo thì xong ở đó. Nhưng phép đo không hoàn hảo: bạn đo hướng lệch đi một chút, nên không có chỗ nào trên cánh đồng làm mọi hướng khớp cùng lúc. Bài toán đổi thành: tìm chỗ mà tổng độ lệch nhỏ nhất.

Và ở đây có hai cách làm, thường bị nhầm là một. Cách thứ nhất là dùng một công thức có sẵn để ra ngay một chỗ khá tốt — nhanh, nhưng công thức ấy tối thiểu hoá một thứ tiện tính chứ không phải chính cái tổng độ lệch mà bạn quan tâm. Cách thứ hai là đứng ở chỗ ấy rồi dịch từng chút theo hướng làm tổng lệch giảm, cho tới khi hết giảm được. Cách thứ hai mới cho câu trả lời đúng, và nó rẻ — vài chục bước tính toán nhỏ.

Nhưng cái quý nhất của cách thứ hai không phải câu trả lời. Khi bạn dừng lại, bạn biết thêm một thứ: mặt đất quanh chỗ bạn đứng *dốc* thế nào. Dốc đứng theo mọi hướng nghĩa là bạn rất chắc chắn — dịch đi một tí là tổng lệch tăng ngay. Có một hướng mà đi dọc theo nó gần như phẳng nghĩa là theo hướng ấy bạn hầu như không biết gì. Cái độ dốc đó chính là thước đo độ tin cậy, và nó có sẵn trong phép tính — ai bỏ bước dịch chuyển thì cũng mất luôn nó, và mất nó thì không còn gì để nói mình chắc chắn tới đâu.

Còn một chuyện nữa, về mấy cái cột. Nếu bạn biết chúng gắn cứng với nhau trên cùng một cái khung — cột này cách cột kia đúng bốn mươi phân — thì bạn nên dùng thông tin đó. Cách sai là tính riêng vị trí theo từng cột rồi lấy trung bình bốn kết quả; làm thế là vứt bỏ hiểu biết về cái khung. Cách đúng là để cả bốn cột cùng kéo về một lời giải duy nhất, vì khi ấy chính khoảng cách giữa chúng trở thành thước đo góc.

Chỗ không rút gọn được: cái giá của chuyện dùng khung là bạn phải *tin* vào khung. Nếu cái khung thật ra không đúng như bản vẽ — cột lệch đi vài milimét — thì cách dùng khung lại tệ hơn cách tính riêng, vì nó ép mọi thứ khớp với một mô hình sai. Ranh giới giữa hai chế độ ấy thì phải đo, không suy ra được.

Một câu: Công thức cho bạn chỗ gần đúng, việc dịch từng chút cho bạn chỗ đúng và cho bạn biết mình chắc tới đâu.

1.10 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — PnP hợp nhất trên toàn bộ tập góc tốt hơn trung bình pose từng tag, và khoảng cách thẳng tăng theo baseline — bị bác bỏ nếu mô phỏng ở mini project cho thấy hai cách cho cùng độ tản mát góc trên constellation trải rộng, với toạ độ 3D chính xác. Đây là mệnh đề tôi dẫn bằng lập luận thông tin (cửa a) và đối chiếu với nguyên lý tổng quát trong [8, §2] (cửa c một phần), và nó *chưa* qua cửa (b). Nó là mệnh đề có hậu quả kiến trúc lớn nhất trong chương nên nên kiểm sớm.

Mệnh đề thứ hai — *hợp nhất trở nên tệ hơn trung bình khi sai số constellation vượt một ngưỡng* — là một dự đoán định tính của tôi dựa trên cơ chế ở mục 5. Nó bị bác bỏ nếu mô phỏng cho thấy hợp nhất vẫn thắng ở mọi mức sai số constellation cho tới khi cả hai đều vô dụng. Nếu vậy thì lời cảnh báo ở mục 5 là quá mạnh và nên được nới.

Mệnh đề thứ ba — *tinh chỉnh cải thiện độ chính xác đáng kể so với lời giải dạng đóng* — bị bác bỏ nếu mô phỏng cho thấy chênh lệch nhỏ hơn nhiều so với σ ở mức nhiễu thực tế. Điều đáng nói: kể cả nếu mệnh đề này bị bác bỏ, lời khuyên “luôn tinh chỉnh” vẫn đứng vững, vì lý do thứ hai — ma trận thông tin miễn phí — không phụ thuộc vào nó.

Cuối cùng, một mệnh đề mà tôi đã cố ý *không* phát biểu: chương này không xếp hạng các thuật toán PnP theo độ chính xác, vì tôi không có dữ liệu so sánh được. Nếu bạn cần xếp hạng ấy cho hệ của mình thì nó phải đo trên chính cấu hình của bạn — và mini project cung cấp khung để làm việc đó.

1.11 Kết nối

Chương này là trực của Phần A: nó nhận mô hình từ hai chương trước và cấp Jacobian cho hai chương sau.

Nối lên trên. A/01-mô-hình-camera-va-distortion cấp hàm chiếu $\hat{u}(\xi)$ và cảnh báo rằng PnP coi K cùng hệ số méo là hằng số đúng — nếu chúng sai thì PnP hội tụ về một pose sai mà không có tín hiệu. A/02-homography-va-hình-học-phang cấp một cách khởi tạo, và cũng cấp lý do vì sao IPPE là lựa chọn tốt hơn phân rã H cho target phẳng.

Nối xuống dưới trong Phần A. A/04-pose-ambiguity-target-phang là hệ quả trực tiếp của mục 8: tối ưu là cực bộ, có hai cực tiểu, và khởi tạo quyết định bạn rơi vào cái nào. A/05-lan-truyen-sai-so-pixel-sang-pose nhận *đúng* ma trận $\mathcal{I}$ từ phương trình chuẩn (1.2) và toàn bộ chương ấy chỉ làm một việc là đọc ý nghĩa vật lý của $\mathcal{I}^{-1}$; Jacobian ở mục 4C là chỗ nguồn gốc của bốn quan hệ tỉ lệ rõ nhất.

Nối xuống Phần B. B/08-corner-refinement cấp Σ_i mà mục 6 cần — không chỉ độ lớn mà cả hướng, vì hiệp phương sai của một góc từ giao hai đường thẳng vốn dị hướng. B/09-multi-marker-constellation là chương biến mục 5 thành thiết kế: nó trả lời đặt tag ở đâu để $\mathcal{I}$ có điều kiện tốt nhất. B/12-calibration-cho-docking nhận cảnh báo ở mục 5 làm lý do bắt buộc cho bước đo constellation bằng bundle adjustment. B/11-uoc-luong-theo-thoi-gian nhận $\mathcal{I}^{-1}$ làm trọng số khi hợp nhất theo thời gian — và nó chỉ làm được điều đó nếu bước tinh chỉnh đã chạy.

Cuối cùng, một nối sang B/13-docking-control: mục 9 khuyến nghị xuất ra ba thứ (pose, hiệp phương sai, ρ) chứ không phải một, và lý do đầy đủ của khuyến nghị ấy nằm ở chương điều khiển — một bộ điều khiển docking cần biết không chỉ “mục tiêu ở đâu” mà cả “tôi tin điều đó tới mức nào” và “có phải tôi đang nhìn nhầm nghiệm không”.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao ba điểm cho bốn nghiệm chứ không phải hai, và vì sao đó là một loại đa nghiệm hoàn toàn khác với hai nghiệm của target phẳng?
2. Nêu ba lý do bắt buộc phải chạy tinh chỉnh sau lời giải dạng đóng, và chỉ ra lý do nào vẫn đứng vững kể cả khi cải thiện độ chính xác là không đáng kể.
3. Trong Jacobian của phép chiếu, cột thứ ba tỉ lệ với X/Z và Y/Z . Điều đó nói gì về khả năng đo chiều sâu của một điểm nằm trên trục quang, và nó nối với công thức nào ở A/05?
4. Một đồng nghiệp trung bình bốn pose từ bốn tag. Giải thích chính xác thông tin gì bị vứt bỏ, và nêu điều kiện mà dưới đó cách làm của họ lại *đúng hơn* cách hợp nhất.

5. Bộ giải của bạn phân kỳ ở đúng một vài tư thế và chạy tốt ở mọi tư thế khác. Nêu nguyên nhân khả dĩ nhất, cơ chế của nó, và cách sửa đúng.
6. Vì sao tổng chi-square trên phần dư đòi Σ_i đúng, và chuyện gì xảy ra với tỉ lệ loại nhầm nếu bạn dùng một σ chung quá nhỏ?
7. Decode sai hướng cho pose sai 90° mà sai số tái chiếu vẫn nhỏ. Vì sao, và cơ chế nào trong chương này phát hiện được nó?

Nguồn

- [1] M. A. Fischler and R. C. Bolles, “Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography,” *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981, mức tin A.
- [2] R. M. Haralick, C.-N. Lee, K. Ottenberg, et al., “Review and Analysis of Solutions of the Three Point Perspective Pose Estimation Problem,” *International Journal of Computer Vision*, vol. 13, no. 3, pp. 331–356, 1994, mức tin A; lịch sử P3P từ Grunert 1841, và phân tích số nghiệm.
- [3] L. Kneip, D. Scaramuzza, and R. Siegwart, “A Novel Parametrization of the Perspective-Three-Point Problem for a Direct Computation of Absolute Camera Position and Orientation,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; P3P dạng đóng, bốn nghiệm, 2011.
- [4] V. Lepetit, F. Moreno-Noguer, and P. Fua, “EPnP: An Accurate $O(n)$ Solution to the PnP Problem,” *International Journal of Computer Vision*, vol. 81, no. 2, pp. 155–166, 2009, mức tin A.
- [5] T. Collins and A. Bartoli, “Infinitesimal Plane-Based Pose Estimation,” *International Journal of Computer Vision*, vol. 109, no. 3, pp. 252–286, 2014, mức tin A; IPPE — hai nghiệm cho target phẳng, là nền của SOLVEPNP_IPPE trong OpenCV.
- [6] G. Terzakis and M. Lourakis, “A Consistently Fast and Globally Optimal Solution to the Perspective-n-Point Problem,” in *European Conference on Computer Vision (ECCV)*, mức tin B; SQPNP, có trong OpenCV, 2020.
- [7] J. Solà, J. Deray, and D. Atchuthan, “A Micro Lie Theory for State Estimation in Robotics,” *arXiv:1812.01537*, 2018, mức tin B; tiền ấn nhưng được dùng rộng rãi làm tài liệu tham chiếu.
- [8] B. Triggs, P. F. McLauchlan, R. I. Hartley, et al., “Bundle Adjustment — A Modern Synthesis,” in *Vision Algorithms: Theory and Practice*, mức tin A; tài liệu tham chiếu chuẩn cho BA, gauge freedom và hiệp phương sai, Springer, 2000, pp. 298–372.
- [9] S. Urban, J. Leitloff, and S. Hinz, “MLPnP — A Real-Time Maximum Likelihood Solution to the Perspective-n-Point Problem,” in *ISPRS Annals of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, mức tin B; PnP có trọng số theo hiệp phương sai từng điểm, 2016.
- [10] S. Agarwal, K. Mierle, et al., “Ceres Solver,” <http://ceres-solver.org>, 2012, mức tin A; phần mềm, không phải bài báo.